

Claude e Claude Code in pratica

Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

Modulo 5 — Skills

Il laboratorio

Vedere scattare non basta

Il metodo di Anthropic: due misure a parte, una baseline, una sessione nuova

- Il **laboratorio** del Modulo 5. Una skill è una cartella con uno `SKILL.md`: istruzioni che Claude aggiunge alla sua dotazione. Qui ne adattate una, su un progetto sintetico del corso, e la provate **come il codice**: il metodo, poi 70 minuti di lavoro
- «Extend Claude with skills»: «Seeing a skill trigger tells you Claude found it, not that it did what you intended.»
- **Due misure, a parte**: «measure separately whether Claude invokes it on the prompts it should, and whether the output matches what you expect when it does.»
- **Con la skill e senza**: «Collect a few realistic prompts, run each one in a fresh session with the skill available and again with it turned off»
- **Senza** vuol dire una copia **senza la cartella della skill**, come nella guida dello standard Agent Skills: «once with the skill and once without it». **Spenta**, con `off` in `skillOverrides`: «Claude doesn't see the skill and /name is hidden from autocomplete», ma «I suoi file restano nella cartella», dice il controllo di M5-D04. È una **garanzia**, non una baseline

Vedere scattare non basta

Il metodo di Anthropic: due misure a parte, una baseline, una sessione nuova

- **La chiamata alla skill fuori dal punteggio**, come in «Test plugins with evals», che parla dei plugin, e il corso usa la stessa regola: «A check like “the skill was invoked” can never pass without the plugin», e gonfierebbe la differenza
- La guida dello standard Agent Skills, «Evaluating skill output quality». **Le asserzioni per ciò che si conta**: «Reserve assertions for things that can be checked objectively.» Stile e impressione «are hard to decompose into pass/fail checks. These are better caught during human review.»
- **Ogni voto con la sua prova**: «recording PASS or FAIL with specific evidence»; «Require concrete evidence for a PASS. Don’t give the benefit of the doubt.» Lo fa uno script: «scripts are more reliable than LLM judgment for mechanical checks»

Vedere scattare una skill dice che Claude l’ha trovata, non che ha fatto ciò che volevate.

Prima le prove

Il metodo, seconda parte: le valutazioni vengono prima delle istruzioni

«Skill authoring best practices»: «Create evaluations BEFORE writing extensive documentation.» I passi, e dove stanno nel laboratorio:

i passi di Anthropic	nel laboratorio
«Run Claude on representative tasks without a Skill.»	`iteration-1`, `without_skill`
«Build three scenarios that test these gaps.»	tre casi, già in `evals.json`
«Measure Claude's performance without the Skill.»	le asserzioni, contate da uno script
«Write minimal instructions: Create just enough content to address the gaps and pass evaluations.»	l'esempio, o il modello di file, adattato
«Iterate: Run evaluations, compare against baseline, and refine.»	`iteration-2`, contro la versione di prima

Un'istanza di Claude scrive, un'altra prova la skill: «Work with one instance of Claude ("Claude A") to create a Skill that is used by other instances ("Claude B").» Qui Claude A è la vostra sessione, Claude B ogni prova

La checklist di Anthropic: «At least three evaluations created»; «Tested with real usage scenarios»

Come si eseguono: «There is not currently a built-in way to run these evaluations. You can create your own evaluation system.» Il nostro è `tools/prove_skill.sh`. Nominati, non richiesti qui: il plugin `skill-creator`, che «automates the comparison loop inside Claude Code», e `claude plugin eval`, per i plugin

Prima le prove

Il metodo, seconda parte: le valutazioni vengono prima delle istruzioni

evals.json, un caso, a capo per la slide: «[…]» un taglio

```
{
  "id": 1,
  "prompt": "Rivedi il pull request di questo branch rispetto a
    main.",
  "expected_output": "Una revisione […] e chiude con un esito.",
  "files": [],
  "assertions": [
    "Cita con il numero tutte le regole violate inserite apposta
      nel pull request",
    "Indica il file e la riga di ogni rilievo",
    "Chiude con un esito: approvabile o da modificare",
    "Non modifica alcun file"
  ]
}
```

- La guida dello standard: «Store test cases in evals/evals.json inside your skill directory»; «Start with 2-3 test cases.»; «Include at least one prompt that tests a boundary condition»
- Le **asserzioni** ci sono già, perché uno script le possa decidere; la guida dice «Add them after you see your first round of outputs». Le vostre, a mano, dopo la baseline, restano «da valutare a mano», fuori dai conteggi

Prima le prove

Il metodo, seconda parte: le valutazioni vengono prima delle istruzioni

- **Una richiesta che non deve attivarla**, in `evals/eval_queries.json`. La guida dello standard: «realistic user prompts labeled with whether they should or shouldn't trigger your skill». «The most valuable negative test cases are near-misses — queries that share keywords or concepts with your skill but actually need something different.» Nella traccia a: «Scrivi il messaggio di commit per le modifiche di questo branch.»
- **È un inizio**: tre casi e una near-miss, dove la stessa guida dice «Aim for about 20 queries: 8-10 that should trigger and 8-10 that shouldn't.», e «Run each query multiple times (3 is a reasonable starting point) and compute a trigger rate». Un articolo di Anthropic, i tre scenari: «Normal operations», «Edge cases», «Out-of-scope requests»
- **Perché «n su 3»**: «Model behavior is nondeterministic — the same query might trigger the skill on one run but not the next.» La pagina sui plugin: «A single run is noisy, so confirm any change at the default three runs before you trust it.» Per questo le demo del modulo dicono «3 su 3»
- **Nel laboratorio, un'esecuzione per caso**: tre sarebbero il triplo delle chiamate. È un campione, e il controllo lo scrive accanto a ogni confronto: «(una sola esecuzione per caso)»

Le tre tracce

Una a testa, sui progetti sintetici del corso

traccia, skill di esempio	i tre casi	che cosa conta il controllo
a · M5-E02 `revisione-pr`	tre pull request del gestionale PHP: un report di esportazione nuovo; una correzione dell'IVA; un'esportazione aggiunta al legacy	le regole inserite apposta, citate con il **numero**; il file e la riga; l'esito
b · M5-E03 `risposta-assistenza`	tre clienti dell'assistenza: una chat sulla linea che va e viene; un'e-mail di uno studio con un firewall nuovo; una chat che chiede la password	le parole da evitare, la premessa, la **forma del canale**, il numero della segnalazione, nessuna password. Il **tono** lo legge una persona
c · M5-E04 `audit-configurazione`	tre apparati della rete, in dialetti diversi: `sw-rovereto-01`, `olt-pergine-01`, `bts-cles-01`	le **categorie** inserite apposta, trovate; la tabella a sei colonne; nessun file modificato

- Si parte dalla **skill di esempio** della traccia, da adattare, o dal modello di file: `riferimenti/skill-di-esempio/` e `riferimenti/template-SKILL.md`. «Skills are the right fit when the agent needs to know how a team does work»: qui le convenzioni, il tono di voce, la checklist
- Tutte sui progetti **sintetici** di CiaspoLink, l'operatore inventato del corso: ciò che Claude Code legge va ad Anthropic, per questo in aula non si lavora su file vostri
- A voce: quale traccia scegliete, e perché?

Il ciclo e l’impronta

Un traguardo per volta, undici chiamate per traccia

```
# dalla cartella esercitazioni; <ID>: M5-E02, M5-E03 o M5-E04
tools/prepara_esercizio.sh <ID>
tools/prove_skill.sh <ID> 1 without      # la baseline: 3 chiamate
tools/prove_skill.sh <ID> copia esempio  # o: copia modello
# in un altro terminale: la skill, in una sessione di Claude Code
tools/prove_skill.sh <ID> 1 with         # la prima versione: 4
# feedback.json, poi il seguito, nella stessa sessione
tools/prove_skill.sh <ID> 2 with         # la seconda versione: 4
tools/verifica.sh <ID>
```

- Ogni prova è un `claude -p` **nuovo**, in una copia pulita dello spazio di lavoro, senza `evals/` e, nella baseline, senza la skill. La guida dello standard Agent Skills: «Each eval run should start with a clean context — no leftover state from previous runs or from the skill development process.»
- Ogni iterazione ha la sua cartella: «Each pass through the full eval loop gets its own iteration-N/ directory.» Stanno in `<skill>-workspace/`, accanto a `.claude/`, fuori da git

Il ciclo e l’impronta

Un traguardo per volta, undici chiamate per traccia

- **Undici chiamate** per traccia: tre senza la skill, quattro per ogni versione, cioè i tre casi e la near-miss. Al massimo tre insieme. Il controllo e la verifica non chiamano il modello
- Le prove su **Sonnet**, effort **low**; la sessione in cui scrivete la skill su Sonnet, effort **medium**. Nessun subagent
- Le prove del docente, un’esecuzione per caso, come dati:

	iterazione 1, 7 chiamate	iterazione 2, 4 chiamate
traccia a	33 turni, 618.208 token, 164,6 s	22 turni, 470.106 token, 91,2 s
traccia b	33 turni, 503.910 token, 107,1 s	20 turni, 320.137 token, 49,5 s
traccia c	69 turni, 1.227.627 token, 249,9 s	34 turni, 615.490 token, 146,6 s

Le righe «Consumo, senza le esecuzioni riusate» del controllo, in tabella: i token di tutti i modelli, la cache compresa

Registrazione del corso, 3 novembre 2026, Claude Code 2.1.317, Sonnet, effort low

- `/usage`: «On Pro, Max, Ultra, Team, and Enterprise it breaks your usage down by skill, subagent, plugin, and MCP server.» Il limite è lo stesso di claude.ai: se si avvicina, raggiungete prima il traguardo in corso

Al lavoro: la baseline

`iteration-1`, `without_skill`: che cosa fa Claude senza la skill, dieci minuti

```
# dalla cartella esercitazioni; o M5-E03, o M5-E04
tools/prepara_esercizio.sh M5-E02
tools/prove_skill.sh M5-E02 1 without
```

- I README, con i passi e i prompt: `m5_skills/s4_skills/m5_e2_laboratorio/`, poi `M5-E02_revisione_pr/` · `M5-E03_risposta_assistenza/` · `M5-E04_audit_configurazione/`
- Tre esecuzioni, una per caso, al massimo tre insieme, ognuna in una **sessione nuova**, senza `evals/` e senza la skill
- Aprite le tre risposte: `<skill>-workspace/iteration-1/eval-1/without_skill/outputs/risposta.md`, e così `eval-2` e `eval-3`

traccia	che cosa dà la versione generica
a	Cita i numeri delle regole? Dice il file e la riga? Chiude con un esito?
b	Comincia con una premessa? Ha la forma del canale? Rispetta il tono di voce dell'assistenza?
c	Trova la checklist da solo? Usa i codici delle categorie? Dice il file e la riga?

- Nella traccia a Claude può scegliere da sé `/code-review`, integrata, e il controllo lo stampa: «Skill chiamata: no (altre: code-review)». È ciò che avete senza una skill vostra
- Le vostre asserzioni, a mano, nell'elenco `assertions` di un caso: restano fuori dai conteggi

Al lavoro: la skill

La skill adattata, in una sessione di Claude Code che resta aperta: venti minuti

```
# dalla cartella esercitazioni: l'esempio, o il modello di file
tools/prove_skill.sh M5-E02 copia esempio
# in un altro terminale, su Sonnet con effort medium
cd ~/corso-claude-code/M5-E02
claude --model sonnet --effort medium
```

- La prima volta Claude Code vi chiede se vi **fidate della cartella**: sì. Poi incollate il **prompt 1** del README: nelle pagine che seguono
- `.claude/` è una cartella **protetta**: quando Claude ci scrive con i suoi strumenti di file, la sessione vi chiede prima. Leggete la modifica, poi: «Yes, and allow Claude to edit files in this project's .claude folder for this session»
- Claude può cambiare il file anche con un **comando**: in una sessione del corso, uno script Python dentro Bash, e Claude Code ha chiesto di approvare ogni comando. **Leggete ogni comando** prima di approvarlo, quello del controllo compreso
- Claude cambia la skill, non i casi di prova né il controllo: tre regole `deny` di `.claude/settings.json` fermano i suoi **strumenti di file**, non un sottoprocesso, per la pagina dei permessi: «like a Python or Node script that opens files itself»
- Lasciate aperta la sessione: è la vostra istanza, «Claude A», e serve all'iterazione. Le prove girano fuori di lì, ognuna in una sessione nuova

Al lavoro: la skill

La skill adattata, in una sessione di Claude Code che resta aperta: venti minuti

il prompt 1 del README, a capo per la slide, 1 di 3

```
<role>
Sei uno sviluppatore del gestionale di CiaspoLink e adatti una
skill di Claude Code per i colleghi che chiedono una revisione.
</role>

<context>
La skill in .claude/skills/revisione-pr/ viene da un esempio del
corso, o dal suo modello, e deve far valere le convenzioni del
team, in progetti/gestionale-php/docs/convenzioni.md: chi la usa
legge la revisione e corregge il codice riga per riga. Come ha
risposto Claude senza la skill è in
revisione-pr-workspace/iteration-1/eval-1/without_skill/
outputs/risposta.md,
e così in eval-2 e in eval-3; ciò che ci si aspetta è nei campi
expected_output di .claude/skills/revisione-pr/evals/evals.json.
</context>
```

Al lavoro: la skill

La skill adattata, in una sessione di Claude Code che resta aperta: venti minuti

il prompt 1 del README, a capo per la slide, 2 di 3

```
<instructions>
1. Leggi il SKILL.md della skill e, se ci sono, i file della sua
   cartella references/.
2. Leggi le tre risposte senza la skill e i campi expected_output,
   e annota che cosa manca alle risposte.
3. Cambia la description e il corpo della skill per ciò che manca.
4. Esegui da solo questo comando, e correggi finché dice 0 errori:
   python3 -B -I controlla_skill.py .claude/skills/revisione-pr
</instructions>

<format>
Al massimo cinque righe: che cosa hai cambiato e perché.
</format>
```

Al lavoro: la skill

La skill adattata, in una sessione di Claude Code che resta aperta: venti minuti

il prompt 1 del README, a capo per la slide, 3 di 3

```
<constraints>
Cambia soltanto i file sotto .claude/skills/revisione-pr/, evals/
escluso. Tieni il SKILL.md nella forma dello standard Agent
Skills, frontmatter e Markdown semplice, con titoli, elenchi,
tabelle e blocchi di codice, sotto le 500 righe. Scrivi ciò che
vale per qualunque pull request del gestionale.
</constraints>
```

- Nelle tracce **b** e **c** il prompt è lo stesso: cambiano il nome della skill e ciò che la skill fa valere, il tono di voce e le procedure, la checklist e l'inventario. Si copia dal **README**, mai dalla slide
- Leggete ciò che scrive, con le parole di Anthropic. La `description`: «Include both what the Skill does and specific triggers/contexts for when to use it.» Il modello della risposta: «Provide templates for output format.» I file di supporto: «Reference supporting files from SKILL.md so Claude knows what each file contains and when to load it». E poco: «Only add context Claude doesn't already have.»

Al lavoro: le prove

`iteration-1`, `with_skill`: i tre casi e la near-miss, dieci minuti

```
# dalla cartella esercitazioni: quattro esecuzioni, ognuna in una
# sessione nuova
tools/prove_skill.sh M5-E02 1 with
```

- Tre file, nella forma della guida dello standard Agent Skills. `grading.json`, per ogni esecuzione: `assertion_results`, cioè `text`, `passed`, `evidence`. Valutare vuol dire «recording PASS or FAIL with specific evidence»
- `timing.json`: `total_tokens` e `duration_ms`. I token sono quelli di `modelUsage`, i subagent compresi: «Use modelUsage, or model_usage in Python, for whole-tree token accounting; the usage field undercounts as soon as nesting occurs.» Quando è diverso, il controllo stampa `usage` accanto
- `benchmark.json`, per l'iterazione: `run_summary`, media e deviazione standard per configurazione, e `delta`: «The delta tells you what the skill costs (more time, more tokens) and what it buys (higher pass rate).»
- «The other JSON files (grading.json, timing.json, benchmark.json) are produced during the eval process — by the agent, by scripts, or by you.» Qui da uno script, che li ricalcola ogni volta dagli stream
- Leggete anche le **risposte**: `iteration-1/eval-*/with_skill/outputs/risposta.md`. I conteggi non le sostituiscono

Al lavoro: le prove

`iteration-1`, `with_skill`: i tre casi e la near-miss, dieci minuti

M5-E02, a capo per la slide: «[…]» segna un taglio

```
Iterazione 1
  eval-1
    without_skill: Skill chiamata: no (altre: code-review); 2 su 4
      asserzioni; 2 turni, 63896 token (usage: 35217), 18,2 s
    with_skill: Skill chiamata: sì; 3 su 4 asserzioni; 6 turni,
      128512 token, 39,5 s
    […]
    with_skill 3, without_skill 2 asserzioni passate: confermata
      (una sola esecuzione per caso)
    […]
  near-miss 1: Skill chiamata: no; dovrebbe attivarla: no;
    2 turni, 39430 token, 8,6 s; […]
  Conteggi: without_skill 5 su 12; with_skill 10 su 12
  Delta, with_skill meno without_skill: pass rate +0,417; tempo
    +5,9 s; token +23957
  […]
(dato) iterazione 1: revisione-pr chiamata 3 su 3 in with_skill
```

Registrazione del corso, 3 novembre 2026, Claude Code 2.1.317, Sonnet, effort low

- La chiamata sta a parte, le asserzioni a parte. La guida: «In early iterations with just 2-3 test cases and single runs, focus on the raw pass counts and the delta»

Al lavoro: l’iterazione

I commenti, che cosa non va, e il seguito nella stessa sessione: venti minuti

iteration-1/feedback.json, a capo per la slide

```
{
  "eval-1": "Elenca le regole, ma due righe non dicono in quale
    file.",
  "eval-2": "",
  "eval-3": "Dice di non unire il ramo, non che cosa spostare in
    src/."
}
```

- Scrivetelo voi, una riga per caso. La guida dello standard: «Record specific feedback for each test case and save it in the workspace». «“The chart is missing axis labels” is actionable; “looks bad” is not.» «Empty feedback means the output looked fine»: la stringa vuota dice che avete letto. Nella traccia b il **tono** sta qui
- Tre segnali: le asserzioni non passate, i commenti, le trascrizioni. «give all three — along with the current SKILL.md — to an LLM and ask it to propose changes». «Read agent execution traces, not just final outputs.»
- Le quattro linee guida: **generalizzare** dai commenti, tenere la skill **snella**, spiegare il **perché**, **raccogliere** il lavoro ripetuto. «When an agent makes a mistake you have to correct, add the correction to the gotchas section.»

Al lavoro: l’iterazione

I commenti, che cosa non va, e il seguito nella stessa sessione: venti minuti

il sintomo	il rimedio, nelle parole di Anthropic e dello standard
non scatta	«Check the description includes keywords users would naturally say». Per `skill-creator`, Claude tende a «undertrigger»: description un po’ «pushy». Ma non le parole di una richiesta: «that’s overfitting»
scatta troppo	«Make the description more specific», o `disable-model-invocation: true`, se la volete solo a mano
scatta quella sbagliata	«your descriptions are too similar — make them more distinct»
salta una regola che deve valere sempre	«move the rule into a hook.»
smette di seguirla	un’istruzione per l’intero compito: «“Run the tests after every edit” rather than “Run the tests”»; il più importante in cima: «Truncation keeps the start of the file»
un file non si trova	«use relative paths from the skill root … Keep file references one level deep from SKILL.md.»
negata dal permesso	nelle prove, con `claude -p` e `dontAsk`, una skill con `allowed-tools` o `disallowed-tools` chiede `Skill(nome)` fra gli strumenti: sta in `prove/config`, e senza quella riga lo strumento si ferma prima. Se c’è, e il reperto resta, lo stream va al docente
il frontmatter non si legge	`claude plugin validate .claude/skills`: «At runtime this skill loads with empty metadata»

Per capire che cosa ha letto: «Ask Claude: “When would you use the [skill name] skill?” Claude will quote the description back.»

Al lavoro: l’iterazione

I commenti, che cosa non va, e il seguito nella stessa sessione: venti minuti

il prompt 2, il seguito, a capo per la slide, 1 di 2

```
<context>
La prima versione della skill è stata provata, e i risultati sono
in revisione-pr-workspace/iteration-1/. I segnali sono tre: le
asserzioni non passate, in eval-1/with_skill/grading.json e così
per eval-2 e eval-3; i commenti di chi ha letto le risposte, in
feedback.json; le trascrizioni, in
eval-1/with_skill/outputs/stream.jsonl e così per gli altri due.
</context>

<instructions>
1. Leggi i tre grading.json, feedback.json e il SKILL.md di
   .claude/skills/revisione-pr/.
2. Per ogni asserzione non passata e per ogni commento, cerca
   nella trascrizione di quel caso, a pezzi con grep, che cosa
   Claude ha letto e che cosa ha saltato.
3. Cambia la skill per le cause che hai trovato.
4. Esegui da solo questo comando, e correggi finché dice 0 errori:
   python3 -B -I controlla_skill.py .claude/skills/revisione-pr
</instructions>
```

Al lavoro: l’iterazione

I commenti, che cosa non va, e il seguito nella stessa sessione: venti minuti

il prompt 2, il seguito, a capo per la slide, 2 di 2

```
<format>
Al massimo cinque righe: che cosa hai cambiato, e quale segnale lo
chiedeva.
</format>

<constraints>
Generalizza dai commenti: la skill serve per ogni pull request,
non per questi tre. Tieni la skill snella: togli le istruzioni che
le trascrizioni mostrano inutili. Spiega il perché di ogni
istruzione. Raccogli in scripts/ il lavoro che ogni esecuzione
rifà da capo. Cambia soltanto i file sotto
.claude/skills/revisione-pr/, evals/ escluso.
</constraints>
```

- `old_skill`, la baseline della seconda iterazione: «When improving an existing skill, use the previous version as your baseline. Snapshot it before editing». La guida la **riesegue**; qui riusa le esecuzioni dell’iterazione 1, per risparmiare la quota del piano: non consuma
- `iteration-N/skill-snapshot/`: una per iterazione, dove la guida ne ha una sola, così ogni coppia di versioni si confronta. Sono due scelte del corso

Al lavoro: la verifica

`iteration-2`: la seconda versione, e che cosa dice la verifica: dieci minuti

```
# dalla cartella esercitazioni
tools/prove_skill.sh M5-E02 2 with
tools/verifica.sh M5-E02
# che cosa è cambiato fra le due versioni
cd ~/corso-claude-code/M5-E02/revisione-pr-workspace
diff -r iteration-1/skill-snapshot iteration-2/skill-snapshot
cd -
```

La guida dello standard: «Rerun all test cases in a new iteration-<N+1>/ directory.» Quattro esecuzioni, ognuna in una sessione nuova; `old_skill` riusa quelle di prima. La fotografia della versione provata sta in `skill-snapshot/`: «Snapshot it before editing»

decide l'uscita: il lavoro è fatto	**si stampa soltanto, come dato**
il file passa `controlla_skill.py`, le regole che si leggono dai file; almeno tre casi in `evals.json`; le due iterazioni, lette dai loro stream; un commento per caso; le due versioni diverse; nella baseline, nessuna chiamata alla skill e la skill non fra le disponibili	la skill chiamata n su 3; la near-miss; le asserzioni, i conteggi e il delta; le parole «confermata», «non distinguibile», «non confermata in questa misura»; turni, token e secondi

Al lavoro: la verifica

`iteration-2`: la seconda versione, e che cosa dice la verifica: dieci minuti

```
tools/verifica.sh M5-E02

[…]
Fatto: la skill passa il controllo del file, le due iterazioni ci
sono, i commenti anche, e le due versioni sono diverse.
VERIFICA OK   M5-E02: fatto.
```

A capo per la slide: «[…]» segna un taglio

Registrazione del corso, 3 novembre 2026, Claude Code 2.1.317, Sonnet, effort low

- `VERIFICA OK` vuol dire che il lavoro è fatto. **Non dice se la skill è buona**: quello lo dite voi, leggendo la tabella e le risposte
- Righe `MANCA`: che cosa manca, con il suo rimedio nel README. Righe `REPERTO`: una garanzia che non regge, per esempio una chiamata «negata dal permesso»
- Chi finisce prima: una **terza iterazione**, `tools/prove_skill.sh <ID> 3 with`, oppure il caricamento su claude.ai di M5-D05